Experiment No. 8

Aim: To implement the A* search algorithm in Python for solving a pathfinding problem in a maze or grid-based environment.

Objectives:

1. To understand the concept of **informed (heuristic-based)** search algorithms.
2. To learn the working of the **A*** (A-Star) algorithm for optimal pathfinding.
3. To implement A* for solving a search problem with real-world constraints.
4. To evaluate how heuristics influence the performance of the search.

Theory:

What is A* Search?

A* is a widely-used informed search algorithm that finds the shortest path from a start node to a goal node using both actual cost from the start and estimated cost to the goal.

It combines the strengths of **uniform-cost search** (which is optimal but slow) and **greedy best-first search** (which is fast but not always optimal).

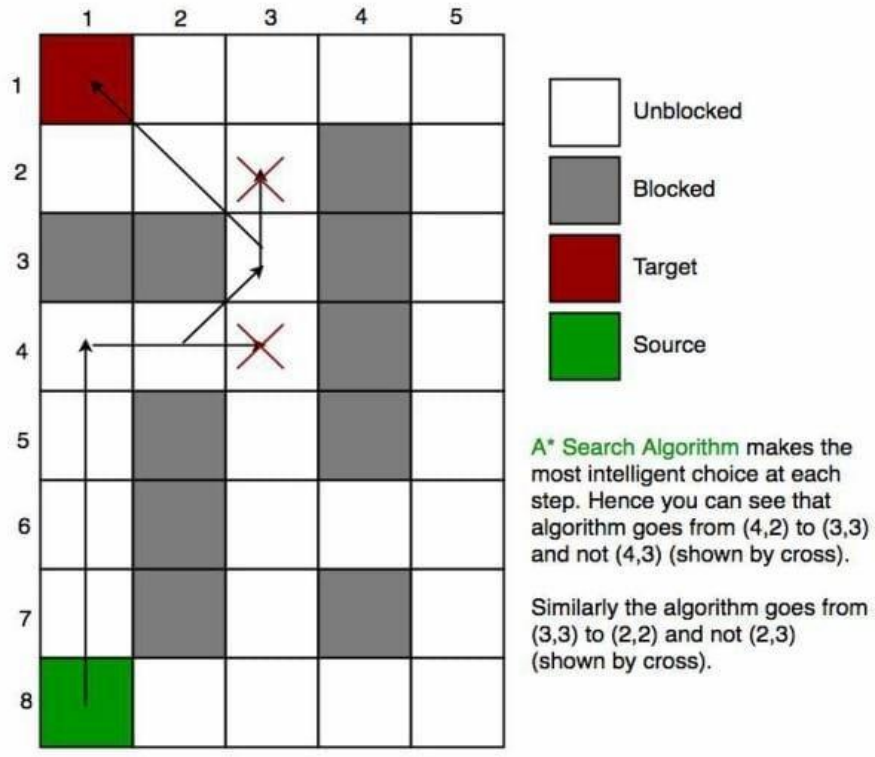


Figure 1: A* Search Algorithm.

A* Cost Function

The total cost function `f(n)` used in A* is:

```
f(n) = g(n) + h(n)
```

Where:

- `g(n)` = Actual cost from the start node to the current node `n`.
- `h(n)` = Heuristic estimate of the cost from `n` to the goal.
- `f(n)` = Total estimated cost of the cheapest solution through node `n`.

Heuristic Function

A heuristic guides the algorithm towards the goal efficiently. For a grid-based environment, common heuristics include:

Manhattan Distance:

```
h(n) = |x1 - x2| + |y1 - y2|
```

Euclidean Distance:

```
h(n) = √[(x1 - x2)² + (y1 - y2)²]
```

The heuristic must be **admissible** (never overestimates the true cost) to ensure A* returns the optimal path.

How A* Works

1. Initialize the open list with the start node.
2. Loop:
    - Pick the node with the lowest `f(n)` from the open list.
    - If it is the goal, stop.

- Else, generate its neighbors and calculate their f, g, and h.
- Add/update them in the open list.

3. Maintain a closed list to avoid revisiting nodes.

Code:

https://github.com/diveshadivarekar/SE-Artificial-Intelligence-Manual-Code/blob/main/a3.py

```
from queue import PriorityQueue

def a_star_search(graph, start, goal, h):
    """
    A* algorithm to find the shortest path from start to goal.

    graph: dictionary where keys are nodes and values are lists of (neighbor, cost) pairs
    start: starting node
    goal: target node
    h: heuristic function estimating cost from any node to goal

    Returns the shortest path and its total cost.
    """

    # Priority queue to pick the next node to explore based on lowest estimated total cost
    open_set = PriorityQueue()
    open_set.put((0, start))  # (priority, node)

    # Keep track of the cost to reach each node from the start
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0  # cost to reach start is zero
```

```
# To remember the best path: which node we came from to reach current node
came_from = {}

while not open_set.empty():
    # Get the node with the lowest total estimated cost (g + h)
    current_f, current = open_set.get()

    # If we reached the goal, build the path to return
    if current == goal:
        path = []
        while current in came_from:
            path.append(current)
            current = came_from[current]  # move backward
        path.append(start)
        path.reverse()  # reverse to get path from start to goal
        return path, g_score[goal]

    # Check neighbors of the current node
    for neighbor, cost in graph[current]:
        # Calculate new cost to neighbor through current
        new_cost = g_score[current] + cost

        # If new cost is better, update the path and cost info
        if new_cost < g_score[neighbor]:
            g_score[neighbor] = new_cost
            priority = new_cost + h(neighbor)  # total cost = cost so far + heuristic estimate
            open_set.put((priority, neighbor))
            came_from[neighbor] = current  # remember where we came from

# If goal was not reached
return None, float('inf')
```

```
# Simple example:

graph = {
   'A': [('B', 1), ('C', 4)],
   'B': [('A', 1), ('C', 2), ('D', 5)],
   'C': [('A', 4), ('B', 2), ('D', 1)],
   'D': [('B', 5), ('C', 1)]
}

# A simple guess of distance from each node to goal D (heuristic)
heuristic = {
   'A': 7,
   'B': 6,
   'C': 2,
   'D': 0
}

def h(node):
   return heuristic[node]

path, cost = a_star_search(graph, 'A', 'D', h)

print("Path:", path)
print("Cost:", cost)
```

Application: Grid Pathfinding

- Environment is modeled as a 2D grid.
- Cells are either **free (0)** or **blocked (1)**.

Input:
```
graph = {
   'A': {'B': 2, 'C': 5},
   'B': {'A': 2, 'D': 2, 'E': 6},
   'C': {'A': 5, 'F': 3},
   'D': {'B': 2},
   'E': {'B': 6, 'F': 1},
   'F': {'C': 3, 'E': 1}
}
heuristic = {
   'A': 7, 'B': 5, 'C': 3, 'D': 3, 'E': 1, 'F': 0
}
start = 'A'
goal = 'F'
```

Output: Path: ['A', 'C', 'F']

Conclusion:

Outcome:

Upon completion of this experiment, students will be able to:

ELO1: Use the A* algorithm to find the best (shortest) path in a graph by combining actual cost and heuristic estimates.

ELO2: Build a working program that applies A* correctly, showing how it selects the most promising nodes during the search.

ELO3: Test and compare A* with other search methods.

Questions:

What is the A* (A star) algorithm?

What is the role of the heuristic function in A*?

What does the evaluation function f(n) represent in A*?

How does A* differ from BFS or DFS?

Give one application of the A* algorithm.